

03-03 模块详细设计与实现说明

系统名称：智慧能源管理系统 系统版本：V4.6.0 文档版本：V1.0 编写日期：2026-09-14 阅读方式：本文以「读代码学架构」方式编写，所有类名、路径、配置键均与仓库一致，建议对照源码阅读。

目录

1. 应用启动与自动装配
 2. MQTT 直采链路（设备 → EMQX → RabbitMQ → 存储/报警）
 3. 医院 IOT 回调链路（HTTP → 鉴权 → MQ → MySQL/TDengine）
 4. 报警引擎实现
 5. 权限与动态菜单实现
 6. 定时任务（Spring Scheduling + XXL-Job）
 7. 报表引擎与模板
 8. 系统集成与数据同步
 9. 业务模块速览（充电桩 / 新能源 / 调度预测 / 数字孪生）
 10. 实战：新增一个 CRUD 业务的完整脚手架
 11. 关键配置项速查表
 12. 本地排障与日志定位指南
-

1. 应用启动与自动装配

1.1 启动类

cpEmsApplication.java（com.cpems 包，admin 模块）：

```
@ComponentScan(basePackages = {"com.cpems", "com.ruoyi"})
@SpringBootApplication
@EnableAsync
@EnableScheduling
public class cpEmsApplication {
    public static void main(String[] args) {
        System.setProperty("spring.devtools.restart.enabled", "false");
        SpringApplication application = new SpringApplication(cpEmsApplication.class);
        application.setApplicationStartup(new BufferingApplicationStart
```

```
up(2048));
    ConfigurableApplicationContext ctx = application.run(args);
    // 启动后打印 active profile、Local/External URL、dbUrl
}
}
```

要点:

- Web 层包根 `com.cpems` 与业务层包根 `com.ruoyi` 都要扫描，这是业务 Bean 能被 Controller 注入的前提；
- `@EnableAsync` 开启异步执行（邮件等）、`@EnableScheduling` 开启 `@Scheduled`（医院离线扫描等）；
- Mapper 不靠注解扫描，而靠 `application.yml` 的 `mybatis-plus.mapperPackage`：
`com.cpems.**.mapper,com.ruoyi.**.mapper,com.autoee.**.mapper。`

1.2 Profile 与 Bean 条件装配

- 根 pom 提供 `local/dev/prod` 三个 profile（`dev` 为默认），profile 同时影响 Maven 资源过滤（`@profiles.active@` 占位符）与 surefire 测试分组；
- 本地运行使用 `local`，对应 `zhurong-ems-admin/src/main/resources/application-local.yml`，自包含全部中间件连接；
- 典型条件装配：XXL-Job 执行器配置类
`com.cpems.job.config.XxlJobConfig` 标注

`@ConditionalOnProperty(prefix = "xxl.job", name = "enabled", havingValue = "true")`

因此 `local` 中 `xxl.job.enabled=false` 时执行器 Bean 不创建，启动不依赖调度中心。

1.3 请求处理链（框架层）

`zhurong-ems-framework` 提供：`Sa-Token` 拦截/鉴权、`security.excludes` 放行清单、`XSS` 过滤器（`xss.enabled=true`，匹配 `/system/*`、`/monitor/*`、`/tool/*`）、全局异常处理器、Web MVC 配置、操作日志 AOP（`@Log`）、防重复提交切面（`@RepeatSubmit`）。业务开发只写 Controller 与 Service，不重复造横切能力。

2. MQTT 直采链路（设备 → EMQX → RabbitMQ → 存储/报警）

2.1 链路总览

电表/水表设备

↓ MQTT publish (QoS1)

EMQX :1883 (electric/voltage、electric/all/+、water/consumption/+ ...)

↓ Paho 客户端订阅

MqttConfiguration → MyMQTTClient → MyMQTTCallback.messageArrived()
↓ rabbitTemplate.convertAndSend("EquipmentTopicExchange", routingKey, Map)

RabbitMQ TopicExchange [EquipmentTopicExchange]

↓ 按 routingKey 绑定到 5 个队列（电压/电流/功率/电能/水量）

MessageListenerContainer (MANUAL ACK, 并发 1) → MyAckReceiver.onMessage()

├ saveDataToDB()	MySQL 业务表
├ saveDataToTDengine() (开关)	@DS("td") EnergyMapper
├ AlarmRule 阈值匹配	RealtimeAlarm 报警记录（状态跃变去重）
├ MailUtils	邮件通知
└ RepairOrder	自动生成维修工单

2.2 逐类说明

① 连接配置：MqttConfiguration.java @Configuration 通过 @Value("\${mqtt.host}") 等读取配置，@Bean 方式创建 MyMQTTClient，连接失败最多重试 10 次、每次间隔 2 秒。

② 客户端封装：MyMQTTClient.java 持有静态 MqttClient (Paho)，connect() 时注册回调 new MyMQTTCallback(this); MqttConnectOptions 开启 setAutomaticReconnect(true)、setCleanSession(true)，并提供 publish(topic, payload, qos, retained)。

③ 订阅与转发：MyMQTTCallback.java 实现 MqttCallbackExtended：

```
@Override
public void connectComplete(boolean reconnect, String serverURI) {
    log.info("MQTT 连接成功, 连接方式: {}", reconnect ? "重连" : "直连");
    myMQTTClient.subscribe(TopicType.ELECTRIC_emsCarson.getInfo(), 1);
    // electric/emsCarson
    myMQTTClient.subscribe(TopicType.ELECTRIC_S.getInfo(), 1);
    // electric/all/+
    myMQTTClient.subscribe(TopicType.ELECTRIC_U.getInfo(), 1);
    // electric/voltage
```

```

        myMQTTClient.subscribe(TopicType.ELECTRIC_I.getInfo(), 1);
// electric/current
        myMQTTClient.subscribe(TopicType.ELECTRIC_W.getInfo(), 1);
// electric/consumption
        myMQTTClient.subscribe(TopicType.ELECTRIC_P.getInfo(), 1);
// electric/power
        myMQTTClient.subscribe(TopicType.WATER_CONSUMPTION.getInfo(), 1);
// water/consumption/+
    }

```

messageArrived 负责协议转换：

- 单值主题 (voltage/current/power/consumption) : payload JSON 对象 → 补 createTime → 原样转发；
- electric/all/{clientId}: payload 为 JSON 数组，按每项 deviceType (electric/voltage 等) 映射到目标 TopicType，拆成多条 {clientId, value, createTime} 转发；
- water/consumption/xxx: 数组逐条转发到水表队列；
- 统一 rabbitTemplate.convertAndSend("EquipmentTopicExchange", TopicType.XXX.getInfo(), maps)，交换机名硬编码为 EquipmentTopicExchange。

主题枚举集中维护：TopicType.java (含 code/info/desc, code 0~4 同时是报警规则的参数标识)。

④ 交换机/队列声明：RabbitExchangeConfig.java 声明持久化 TopicExchange EquipmentTopicExchange 与 5 个队列（队列名即主题字符串），用 routing key —— 绑定。> 注意：该类中 P/W 两个 Binding 方法存在交叉绑定的历史写法（PExchangeMessage 绑 W 队列、WExchangeMessage 绑 P 队列），改动消息路由时务必先读源码确认，不要「想当然」修正。

⑤ 监听容器：MessageListenerConfig.java SimpleMessageListenerContainer，并发 1，AcknowledgeMode.MANUAL，setQueueNames(U,I,W,P,Water)，消息监听器为 MyAckReceiver。

⑥ 消费落库与联动：MyAckReceiver.java 实现 ChannelAwareMessageListener，核心流程：

```

ObjectInputStream ois = new ObjectInputStream(new ByteArrayInputStream(
    message.getBody()));
Map<String,Object> msgMap = (Map<String,Object>) ois.readObject();
BigDecimal value = new BigDecimal(msgMap.get("value").toString());
String clientId = msgMap.get("clientId").toString();
String createTime = msgMap.get("createTime").toString();

```

```
String routingKey = message.getMessageProperties().getReceivedRoutingKey();
```

```
saveDataToDB(clientId, routingKey, createTime, value);  
if (tdDbEnabled) { saveDataToTDengine(clientId, routingKey, createTime,  
    value); }  
// 之后按 routingKey 分支做 AlarmRule 评估
```

注意：该链路使用 **Java 原生序列化** (ObjectInputStream)，消息体必须是序列化 Map，模拟器侧只负责发到 MQTT，JSON→Map 的转换在 MyMQTTCallback 完成、经 AMQP 消息转换器序列化后再被消费。新增主题消费时保持同一消息模型。

⑦ **TDengine 时序落库**： - 实体 Energy.java: @TableName("energy"), 字段 ts(Timestamp)、client_id、val(float)、TAG type; - Mapper EnergyMapper.java: 类级 @DS("td") 切换数据源; - 总开关 @Value("\${spring.datasource.dynamic.datasource.td.enabled:false}"), local 默认 false。

3. 医院 IOT 回调链路 (HTTP → 鉴权 → MQ → MySQL/TDengine)

医院子域是「加性开发」的标准范本，代码位于： - Web: zhurong-ems-admin 的 com.cpems.web.controller.hospital - 业务: zhurong-ems-system 的 com.ruoyi.system.hospital (config / service(+impl) / mq / mapper / domain(+enums) / bo / vo / dto / constant / energy)

3.1 链路总览

外部 IOT 平台

| POST /hospital/callback/data (X-IOT-Token / X-IOT-Sign)



HospitalCallbackController (Sa-Token 排除路径)



IHospitalDataIngestService.handleCallback

| HospitalCallbackLog 先落「接收日志」

| IotCallbackAuthService Token + IP 白名单 + HMAC-SHA256 签名

| IotDataParser 报文 → List<StandardDataPoint> (iot_device_id 绑定映射)

| RabbitTemplate JSON 投递 hospital.topic.exchange / hospital.device.data

| 更新回调日志状态/点数/耗时



```

HospitalDeviceDataConsumer (@RabbitListener, MANUAL)
├─ HospitalDeviceDataMapper.insertBatch()    MySQL hospital_device_data
├─ HospitalTdMapper.insertPoint() (开关)    TDengine 超表 hospital_device_data
├─ IHospitalAlarmEvalService.evalPoints()    阈值报警
└─ basicAck; 异常 basicNack(requeue=false) → 死信

```

▼
HospitalDeviceDataRetryConsumer (死信重试队列, 最多 3 次再投递, 超限人工排查)

3.2 入口与免登录配置

HospitalCallbackController.java:

```

@RestController
@RequestMapping("/hospital/callback")
public class HospitalCallbackController {
    @PostMapping("/data")
    public R<String> receive(@RequestHeader(value="X-IOT-Token", required=false) String token,
                             @RequestHeader(value="X-IOT-Sign", required=false) String sign,
                             @RequestBody IotCallbackRequest body,
                             HttpServletRequest request) {
        String result = dataIngestService.handleCallback(token, getClientIp(request), sign, body, body==null?null:body.getMsgId());
        return "OK".equals(result) ? R.ok("OK") : R.fail(result);
    }
}

```

/hospital/callback/** 配置在 application.yml 的 security.excludes 中 (免 Sa-Token), 但应用层有独立鉴权, 不是匿名开放接口。客户端 IP 解析支持 X-Forwarded-For/X-Real-IP 多级代理取首段。

3.3 回调报文模型 (dto)

IotCallbackRequest.java:

```

{
  "msgId": "msg-001",
  "timestamp": "2026-09-03 10:00:00",
  "devices": [
    { "deviceId": "iot-device-001",
      "points": [
        {"metric": "power", "value": 12.34, "ts": "2026-09-03 10:00:00",
        "quality": 0}
      ] }
  ] }

```

```
    ]  
}
```

内嵌 Device (IOT 平台设备 ID, 需与 hospital_device.iot_device_id 绑定) 与 Point (指标编码对应 hospital_metric_def.metric_code, value 可为数值或状态码)。

3.4 标准化数据点 (跨 MQ 载体)

StandardDataPoint.java (Serializable) :

字段	含义
deviceId	本系统设备 ID (hospital_device.id)
deviceCode	设备编号 (同时作 TDengine TAG 与子表名素材)
metricCode	指标编码
value (BigDecimal)	数值型值
strValue (String)	状态/字符串型值
ts (Date)	采集时间
quality	0 正常 / 1 异常

3.5 鉴权

lotCallbackAuthServiceImpl.java, checkFailReason 三道关卡, 任一不过返回中文失败原因:

1. **Token**: X-IOT-Token 与配置 hospital.iot.auth-token (默认 hospital-iot-2026) 常量比对;
2. **IP 白名单**: hospital.iot.ip-whitelist 支持逗号/分号分隔, 空表示不限;
3. **签名 (可选)**: sign-enabled=true 且配置了 sign-secret 时, 校验 X-IOT-Sign:

```
hex(HMAC_SHA256(secret, timestamp + "\n" + msgId))
```

使用 Hutool HMac(HmacAlgorithm.HmacSHA256, secret.getBytes(UTF_8)).digestHex(...), 大小写不敏感比较。

配置载体: HospitalIotProperties.java

```
(@ConfigurationProperties("hospital.iot"))。
```

3.6 解析标准化

lotDataParserImpl.java:

- 遍历 devices[].points[];
- hospitalDeviceService.queryDeviceIdByIotDeviceId(iotId) 做外部 ID → 本系统 ID 绑定, 未绑定设备跳过并 **warn**;
- 时间兼容 yyyy-MM-dd HH:mm:ss 与 ISO8601 (Hutool DateUtil.parse) , 为空取当前时间;
- 值可转 BigDecimal 入 value, 否则入 strValue; quality 空默认 0。

3.7 接入服务 (编排 + 回调日志)

HospitalDataIngestServiceImpl.java 是典型的「先记账、再处理、终态回写」编排:

```
Long logId = callbackLogService.insertLog(logEntity);           // 先落
接收日志
String failReason = authService.checkFailReason(...);          // 鉴权
List<StandardDataPoint> points = iotDataParser.parse(request); // 解析
String payload = JSON.toJSONString(points);
rabbitTemplate.convertAndSend(iotProperties.getExchange(),
                              iotProperties.getRoutingKey(), payload);
// 投递
// 回写日志: status (成功/鉴权失败/处理失败)、deviceCount、pointCount、costTime、errorMsg
```

回调日志实体 HospitalCallbackLog, 支持后台「回调日志」页面追溯每次推送 (HospitalCallbackLogController) 。

3.8 MQ 声明: 主队列 + 死信重试

HospitalRabbitConfig.java:

- TopicExchange hospital.topic.exchange (durable) ;
- 主队列 hospital.device.data.queue, 参数:
args.put("x-dead-letter-exchange", iotProperties.getExchange());
args.put("x-dead-letter-routing-key", "hospital.device.data.retry");
- 绑定 routing key hospital.device.data;
- 重试队列 hospital.device.data.retry.queue + 绑定; 常量
MAX_RETRY_TIMES = 3。
- 与老设备链路 EquipmentTopicExchange 物理隔离。

3.9 主消费者

HospitalDeviceDataConsumer.java:

```
@RabbitListener(queues = "${hospital.iot.queue}", ackMode = "MANUAL")
public void onMessage(String payload, Channel channel,
                      @Header(AmqpHeaders.DELIVERY_TAG) long deliveryTag) {
    try {
        List<StandardDataPoint> points = JSON.parseArray(payload, StandardDataPoint.class);
        List<HospitalDeviceData> list = new ArrayList<>(points.size());
        for (StandardDataPoint p : points) {
            list.add(toEntity(p));
            if (tdDbEnabled) { saveToTdengine(p); }
        }
        deviceDataMapper.insertBatch(list);
        alarmEvalService.evalPoints(points);
        channel.basicAck(deliveryTag, false);
    } catch (Exception e) {
        channel.basicNack(deliveryTag, false, false); // requeue=false
        → 死信队列
    }
}
```

TDengine 写入: insert into \${子表} using hospital_device_data tags(?,?) (ts,metric_value,quality) values(?,?,?), 子表名由 deviceCode + "_" + metricCode 经 [^a-zA-Z0-9_]→_ 清洗生成; 任何 TD 异常仅日志、不阻断 MySQL 主链路。

3.10 重试消费者

HospitalDeviceDataRetryConsumer.java: 从消息头 x-retry-count 读次数; <3 则复制消息头、次数 +1 重新 rabbitTemplate.send 到主队列并 ack; ≥3 次记录 error 日志 (保留 payload) 并 ack, 等待人工排查; 自身异常 basicNack(requeue=true) 回重试队列, 避免丢失。

3.11 数据权限

HospitalDataScopeServiceImpl.java:

- isUnrestricted(): 超级管理员/拥有全部权限时返回 true, 查询不加院区条件;
- resolveAccessibleAreas(): 委托平台 ISysDataScopeService 解析当前用户可见院区 (dept) 集合;

- 各业务 Service（设备、报警、工作量、能源分析）在 buildQueryWrapper 中把可见集合作为 area_id 过滤条件。

4. 报警引擎实现

系统有两套报警实现，理解差异后再改动。

4.1 设备直采报警：MyAckReceiver（老链路）

- 规则表 alarm_rule（AlarmRule/AlarmRuleMapper），字段含 alarm_switch（0 启用）、param_name（对应 TopicType.code：0 电压/1 电流/2 功率/3 电能/4 水量）、condition1/2、threshold_value1/2；
- 条件类型枚举 AlarmConditionType：G/L/E/GE/LE（大于/小于/等于/大于等于/小于等于）；
- 仅对电类主题分支评估，**双阈值**逐条判断；
- 去重策略：getLast(clientId, paramName) 取最近一条同参数报警，仅在状态跃变（上次未超阈值、本次超阈值）时 insertAlarm，避免同一条异常持续刷屏；
- 联动：RealtimeAlarm 落实时报警表、MailUtils 发邮件通知、RepairOrderMapper 自动开维修工单；
- 历史/规则管理：com.cpems.web.controller.alarm 下 RealtimeAlarmController、AlarmRuleController、AlarmHistoryController。

4.2 医院报警：HospitalAlarmEvalService（新链路，推荐范式）

HospitalAlarmEvalServiceImpl.java：

THRESHOLD（阈值）规则——数据驱动、同步评估：

```
public void evalPoints(List<StandardDataPoint> points) {
    List<HospitalAlarmRule> rules = ruleMapper.selectList(
        new LambdaQueryWrapper<HospitalAlarmRule>()
            .eq(HospitalAlarmRule::getRuleType, HospitalConstants.ALARM_RULE_THRESHOLD)
            .eq(HospitalAlarmRule::getStatus, "0"));
    for (StandardDataPoint p : points) {
        for (HospitalAlarmRule rule : rules) {
            if (!matchDevice(rule, p.getDeviceId(), null)) continue;
            if (!StrUtil.equals(rule.getMetricCode(), p.getMetricCode())) continue;
            if (hitCondition(rule.getRuleCondition(), p.getValue(), rul
```

```

e.getThresholdValue())) {
    fireThreshold(rule, p); // 存在同规则未处理记录则不重复建
    单
    }
    }
}
}

```

OFFLINE（离线）规则——时间驱动、定时扫描：

```

@Scheduled(fixedDelay = 300000) // 每 5 分钟
public void scanOffline() { ... }

```

通过 `deviceDataMapper.selectMaxTsByDeviceIds(null)` 取各设备最后数据时间，超过规则 `offlineTimeoutMin`（空则默认 30 分钟，配置默认值见 `hospital.iot.monitor-offline-minutes`）则 `fireOffline` 建单；数据恢复后 `closeOffline` 自动结束报警。报警记录 `HospitalAlarmRecord`，通知同样复用 `MailUtils`。整个评估方法 `try/catch` 兜底——**报警异常不影响消费 ACK**。

管理端： `HospitalAlarmRuleController`（规则 CRUD，权限 `hospital:alarmRule:*`）、 `HospitalAlarmRecordController`（记录处理）。

5. 权限与动态菜单实现

5.1 登录签发（后端）

`SysLoginController.java`（无类级 `@RequestMapping`，路径为根路径）：

接口	说明
POST /login	@SaIgnore，校验验证码后 SysLoginService.login 完成密码校验 (BCrypt) 与 Sa-Token 登录，返回 {token}
POST /logout	注销当前会话
GET /getInfo	返回 user、roles（角色标识集合）、 permissions（菜单权限码集合），数据 取自 LoginHelper.getLoginUser()
GET /getRouters	动态菜单核心： menuService.selectMenuTreeByUserId (userId) → menuService.buildMenus(menus) 组装

接口	说明
	RouterVo 树
POST /smsLogin、/xcxLogin、 /mockLogin	短信/小程序/模拟登录
Sa-Token 配置 (application.yml) : token 名 Authorization、前缀 Bearer、有效期 15 天、JWT 模式 (sa-token-jwt, 密钥 jwt-secret-key)、仅从 Header 读取。	
菜单管理: SysMenuController (/system/menu) 维护目录/菜单/按钮三类数据, 按钮类型记录权限标识 (perms) , 如 hospital:device:list。	

5.2 前端鉴权链路

1. src/permission.js 注册全局前置守卫: 无 token → 跳 /login (白名单 /login、/register、/applyAccount) ; 有 token 且未加载角色时依次 GetInfo → GenerateRoutes → router.addRoutes;
2. store/modules/user.js: Login 存 token, GetInfo 存 roles/permissions/username/userId/avatar;
3. store/modules/permission.js: GenerateRoutes 请求 /getRouters 后由 filterAsyncRouter 把后端 component 字符串映射为组件:

```
if (route.component === 'Layout') route.component = Layout
else if (route.component === 'ParentView') route.component = ParentView
else if (route.component === 'InnerLink') route.component = InnerLink
else route.component = loadView(route.component)
// loadView: dev 用 require, prod 用 () => import(`@/views/${view}`)
```

因此菜单表 component 字段必须与 src/views/ 下文件相对路径严格一致 (如 hospital/device/index) 。独立全屏页 /hospital/screen 由 stripFullScreenRoutes 从动态路由剔除 (在 constantRoutes 中定义、菜单以新标签页打开) 。

4. 按钮权限: 指令 v-hasPermi / v-hasRole (src/directive/ 注册, 实现于 directive/permission/), 无权限直接移除 DOM; JS 中用 plugins/auth.js 的 hasPermi/hasPermiOr/hasRole;
5. 请求注入: utils/request.js 自动加 Authorization: Bearer <token>; 响应码 401 触发重新登录弹窗。

6. 定时任务 (Spring Scheduling + XXL-Job)

6.1 Spring @Scheduled (随应用启动)

启动类 @EnableScheduling, 轻量定时直接注解开发, 如医院离线扫描:

```
@Scheduled(fixedDelay = 300000)
public void scanOffline() { ... }
```

适用于单机、无运维界面需求的周期任务。

6.2 XXL-Job (分布式调度)

执行器: zhurong-ems-job 模块

- 配置类 XxlJobConfig.java + 属性
com.cpems.job.config.properties.XxlJobProperties;
- @ConditionalOnProperty(prefix="xxl.job", name="enabled",
havingValue="true"), 注册 XxlJobSpringExecutor (appname、port
9101、logpath ./logs/xxl-job) ;
- JobHandler 用 @XxlJob("handlerName") 写在业务模块, 例如报表任务
ReportTaskServiceImpl; local 默认关闭执行器。

调度中心: zhurong-ems-extend/zhurong-ems-xxl-job-admin (独立 Spring Boot 应用, 启动类 com.xx1.job.admin.XxlJobAdminApplication, XXL-Job 2.3.1 官方控制台), 前端地址约定 http://localhost:9100/xxl-job-admin (见 .env.development 的 VUE_APP_XXL_JOB_ADMIN)。调度中心依赖 xxl_job 库, 初始化 SQL 见 docker/mysql/init_xxl_job_tables.sql、tables_xxl_job.sql。

监控: 同模块下 zhurong-ems-monitor-admin (Spring Boot Admin, MonitorAdminApplication), local 中
spring.boot.admin.client.enabled=false。

联调步骤: 建 xxl_job 库 → 启动 xxl-job-admin → 放开 xxl.job.enabled 与
admin-addresses → 执行器自动注册 → 控制台配置任务 (Cron、handler、参数)。

7. 报表引擎与模板

位于 admin 模块 com.cpems.web.controller.report:

- ReportTemplateController.java: /system/report/template, 报表模板 CRUD (数据源/SQL 或数据集合、参数定义、版式) ;
- ReportEngineController.java: /system/report/engine, 模板渲染、预览、导出 (Excel/PDF, PDF 由 OpenPDF 1.3.30 输出) ;
- 异步/周期出报表由 XXL-Job 调度 ReportTaskServiceImpl (@XxlJob) 生成并落库/推送;
- 前端 API: src/api/report/template.js、src/api/report/engine.js。

开发新增报表优先用「模板 + 参数 + 引擎渲染」方式, 避免为每张报表写专用导出接口; 模板内 SQL 必须由开发维护、参数化绑定, 禁止把终端用户输入拼接进模板 SQL。

8. 系统集成与数据同步

com.cpems.web.controller.system 下一组集成接口:

Controller	路径前缀	职责
SyncEngineController	/system/integration/sync/engine	同步引擎开关与手动触发: execute/{taskId}、executeAll、start、stop、status
SysSyncTaskController	/system/...	同步任务配置 (源/目标接口、Cron、字段映射、启停)
SysSyncExecutionController	—	同步执行记录 (开始/结束时间、状态、条数、错误信息)
SysInterfaceConfigController	—	外部接口配置 (URL、鉴权、超时)
SysIntegrationConfigController	—	集成总配置
SysApiInterfaceController	—	API 接口目录定义

核心服务 ISyncEngineService: 按启用任务调度执行同步、写入执行记录; 任务/执行实体为 com.ruoyi.system.domain.SysSyncTask、SysSyncExecution, 接口配置为 SysInterfaceConfig、SysIntegrationConfig、SysApiInterface。设计文档

参见 docs/技术文档/系统集成模块技术文档.md 与第三阶段对接方案。前端 API 在 src/api/（集成相关页面）。

9. 业务模块速览（充电桩 / 新能源 / 调度预测 / 数字孪生）

所有业务 Controller 均在 com.cpems.web.controller.*，路径与包结构如下：

业务域	包 / 路径前缀	主要 Controller
充电桩站	controller.chargingStation, /chargingStation/*	充电站 ChargingStationController、充电桩 ChargingPileController、品牌/厂商 BrandController/ChargingBrandController、商 户 MerchantController、 型号 ModelController、 计价参数/策略 ChargingPriceParam/ChargingPriceStrategyController、订单 OrderInfoController、 占位订单 OccupancyOrderController
新能源	controller.newenergy, /newenergy/*	光伏电站 PvStationController、 储能系统 StorageSystemController、储能电池 StorageBatteryController、能量存储 EnergyStorageController、微网 MicroGridController、 虚拟电厂 VirtualPowerPlantController、新能源统计 NewEnergyStatisticsController

业务域	包 / 路径前缀	主要 Controller
调度与预测	controller.dispatch, /system/dispatch/*	负荷预测 LoadForecastController 、需量预测 EnergyDemandForecastController (/forecast) 、电价预测 EnergyPriceForecastController (/priceForecast) 、气象 WeatherForecastController、预测模型配置 ForecastModelConfigController、优化算法/方案 OptimizationAlgorithm/OptimizationSchemeController、配电网/节点 DistributionNetwork/NetworkNodeController、 调度指令/计划/协同/评价 DispatchCommand/EnergyPlan/ProductionEnergyCollaboration/Evaluation(Report)Controller
数字孪生	controller.digitaltwin, /system/digitaltwin/*	孪生设备 TwinDeviceController (/device) 、3D 模型 ThreeDModelController (/model) 、能流 EnergyFlowController (/flow)
其他	—	能耗 energy/EnergyController、碳排放 carbon/CarbonController、水务 water/WaterController 、计量器具与检定 metering/*、巡检/排班/ 工单 inspection/*、设备

业务域	包 / 路径前缀	主要 Controller
		/网关 equipment/*、指标 quota/QuotaConfigController、视频 camera/CameraConfigController、分项分析 itemizeanalysis、数据查询 dataQuery/DataQueryController

开发要点：这些域共享同一套平台底座（Sa-Token 权限码、@Log、MyBatis-Plus、逻辑删除、雪花 ID、统一 R/TableDataInfo、前端 request/字典/i18n），新增业务必须沿用，不允许引入第二套权限或 HTTP 客户端体系。

10. 实战：新增一个 CRUD 业务的完整脚手架

需求示例：新增「医院耗材台账」业务 hospital_supplies。严格按下列顺序执行，每一步都有本仓库现成参照物。

步骤 1：建表 SQL

在 数据库脚本/ 新增 hospital_supplies.sql，或追加到下一迭代 hospital_menu_mX.sql 同批发布。表名加模块前缀、字段加注释、带审计列与 del_flag：

```
CREATE TABLE IF NOT EXISTS `hospital_supplies` (  
  `id` BIGINT NOT NULL COMMENT '主键(雪花)',  
  `supplies_name` VARCHAR(128) NOT NULL COMMENT '耗材名称',  
  `supplies_code` VARCHAR(64) NOT NULL COMMENT '耗材编号',  
  `spec` VARCHAR(128) COMMENT '规格型号',  
  `stock_num` INT DEFAULT 0 COMMENT '库存数量',  
  `status` CHAR(1) DEFAULT '0' COMMENT '状态(0 正常 1 停用)',  
  `del_flag` CHAR(1) DEFAULT '0' COMMENT '删除标志(0 存在 2 删除)',  
  `create_by` VARCHAR(64) COMMENT '创建者',  
  `create_time` DATETIME COMMENT '创建时间',  
  `update_by` VARCHAR(64) COMMENT '更新者',  
  `update_time` DATETIME COMMENT '更新时间',  
  `remark` VARCHAR(512) COMMENT '备注',  
  PRIMARY KEY (`id`),
```

```
    UNIQUE KEY `uk_supplies_code` (`supplies_code`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COMMENT='医院耗材台账';
```

需要字典/菜单的同步准备 sys_dict_type/sys_dict_data 与 sys_menu insert (权限码 hospital:supplies:list/query/add/edit/remove/export), 参照 hospital_menu_m4.sql 写法。

步骤 2: 实体 domain

```
com.ruoyi.system.hospital.domain.HospitalSupplies: @Data
@Table("hospital_supplies") @ExcelIgnoreUnannotated, @TableId("id")
(全局 ASSIGN_ID 可省略 type)、@TableLogic 标注 delFlag, 导出字段加
@ExcelProperty。参考 HospitalDevice。
```

步骤 3: BO / VO

- bo/HospitalSuppliesBo: 查询条件 + 新增/编辑字段, 加 @NotNull(groups=EditGroup.class) 等分组校验;
- vo/HospitalSuppliesVo: 出参 (可加字典翻译属性, 不暴露 delFlag)。参考 HospitalDeviceBo/HospitalDeviceVo。

步骤 4: Mapper + XML

```
public interface HospitalSuppliesMapper extends BaseMapperPlus<Hospital
SuppliesMapper,
Hospita
lSupplies,
Hospita
lSuppliesVo> {
    // 复杂查询才写 XML: resources/mapper/hospital/HospitalSuppliesMappe
r.xml
}
```

分页/详情直接用 selectVoPage/selectVoById/selectVoList, 无需 XML。

步骤 5: Service 接口 + 实现

- 接口 IHospitalSuppliesService: queryPageList(bo, pageQuery)、queryById、insertByBo、updateByBo、deleteWithValidByIds;
- 实现 HospitalSuppliesServiceImpl: @RequiredArgsConstructor @Service, 持有 private final HospitalSuppliesMapper baseMapper;, buildQueryWrapper(bo) 集中拼条件; 列表查询经 IHospitalDataScopeService 加院区范围; TableDataInfo.build(page) 返回。参考 HospitalDeviceServiceImpl。

唯一性校验（如 supplies_code 重复）抛 ServiceException("耗材编号已存在");
多表写入加 @Transactional(rollbackFor = Exception.class)。

步骤 6: Controller

com.cpems.web.controller.hospital.HospitalSuppliesController, 完全套用
HospitalDeviceController 模板:

```
@Validated
@RequiredArgsConstructor
@RestController
@RequestMapping("/hospital/supplies")
public class HospitalSuppliesController extends BaseController {

    private final IHospitalSuppliesService hospitalSuppliesService;

    @SaCheckPermission("hospital:supplies:list")
    @GetMapping("/list")
    public TableDataInfo<HospitalSuppliesVo> list(HospitalSuppliesBo bo,
        PageQuery pageQuery) {
        return hospitalSuppliesService.queryPageList(bo, pageQuery);
    }

    @SaCheckPermission("hospital:supplies:query")
    @GetMapping("/info/{id}")
    public R<HospitalSuppliesVo> getInfo(@NotNull(message = "主键不能为空") @PathVariable Long id) {
        return R.ok(hospitalSuppliesService.queryById(id));
    }

    @SaCheckPermission("hospital:supplies:add")
    @Log(title = "医院耗材", businessType = BusinessType.INSERT)
    @RepeatSubmit()
    @PostMapping()
    public R<Void> add(@Validated(AddGroup.class) @RequestBody Hospital
        SuppliesBo bo) {
        return toAjax(hospitalSuppliesService.insertByBo(bo));
    }

    @SaCheckPermission("hospital:supplies:edit")
    @Log(title = "医院耗材", businessType = BusinessType.UPDATE)
    @RepeatSubmit()
    @PutMapping()
    public R<Void> edit(@Validated(EditGroup.class) @RequestBody Hospital
        alSuppliesBo bo) {
        return toAjax(hospitalSuppliesService.updateByBo(bo));
    }
}
```

```

        @SaCheckPermission("hospital:supplies:remove")
        @Log(title = "医院耗材", businessType = BusinessType.DELETE)
        @DeleteMapping("/{ids}")
        public R<Void> remove(@NotEmpty(message = "主键不能为空") @PathVariable Long[] ids) {
            return toAjax(hospitalSuppliesService.deleteWithValidByIds(Arrays.asList(ids)));
        }
    }
}

```

启动后接口自动进入 springdoc 「2.系统模块」 分组。

步骤 7：菜单与权限标识

执行菜单 SQL（目录挂医院模块，组件路径 hospital/supplies/index，perms 与注解完全一致），给角色分配后重新登录验证 /getRouters。

步骤 8：前端 API 与页面

- 新建 src/api/hospital/supplies.js，导出 listSupplies/getSupplies/addSupplies/updateSupplies/delSupplies（复制 api/hospital/device.js 改路径）；
- 新建 src/views/hospital/supplies/index.vue：搜索表单 + el-table + Pagination + RightToolbar + 新增/编辑弹窗；按钮加 v-hasPermi="['hospital:supplies:add']"；字典用 dicts:[...] + <dict-tag>；所有文案 \$t('hospital.supplies.xxx') 并补 4 语言 key；属性绑定全部用 :prop；
- 删除走 this.\$modal.confirm，提交用 this.\$modal.loading；
- 菜单 component 配 hospital/supplies/index 后即可动态加载。

步骤 9：自测清单

建表导入 → 后端 Swagger 五类接口通过（含无权限 403/无 token 401）→ 菜单可见/按钮按权限显隐 → 列表分页/搜索/数据权限正确 → 新增防重复提交 → 逻辑删除后列表不可见但库中 del_flag=2 → 导出（如需要）。

11. 关键配置项速查表

配置键	默认/示例值	说明	位置
server.port	启动参数 8088	本地端口（yml 默	application.yml /

配置键	默认/示例值	说明	位置
		认 8080, 启动参数覆盖)	Run Config
server.servlet.context-path	/autoee-iot-ems	本地统一前缀	启动参数
spring.profiles.active	local	环境 profile	启动参数
spring.datasource.dynamic.datasource.master.url	jdbc:mysql://localhost:3306/autoee_ems	主库	application-local.yml
spring.datasource.dynamic.datasource.td.enabled	false	TDengine 总开关	启动参数显式关闭
spring.redis.host/port/database/password	localhost/6379/13/difyai123456	Redis (db13, 必须有密码)	application-local.yml
redisson.singleServerConfig.*	同上	Redisson 独立声明, 密码需同步	application-local.yml
spring.rabbitmq.*	localhost:5672, rabbitmq/rabbitmqpassword, vhost admin_vhost	RabbitMQ	application-local.yml
mqtt.host/username/password	tcp://localhost:1883, admin/public	EMQX 连接	application-local.yml
mqtt.clientId/queue/timeout/keepalive	zhurong-ems-local-client/1/120/360	Paho 客户端参数	application-local.yml
xxl.job.enabled	false (local)	执行器注册开关	application-local.yml
xxl.job.admin-addresses	http://localhost:8080/xxl-job-admin	调度中心地址 (联调时改实际端口)	application-local.yml
spring.boot.admin.client.enabled	false	SBA 客户端注册	application-local.yml
sa-token.token-name / token-prefix	Authorization / Bearer	请求头	application.yml
sa-token.timeout	1296000 (15 天)	token 有效期	application.yml
security.excludes	静态资源、api-docs、actuator、	免登录路径	application.yml

配置键	默认/示例值	说明	位置
	/hospital/callback/**		
hospital.iot.auth-token	hospital-iot-2026	回调 X-IOT-Token	application.yml
hospital.iot.sign-enabled/sign-secret	false / 空	HMAC 签名开关与密钥	application.yml
hospital.iot.monitor-offline-minutes	30	设备离线判定阈值 (分钟)	application.yml
hospital.iot.exchange/queue/routing-key	hospital.topic.exchange / hospital.device.data.queue / hospital.device.data	医院 MQ 拓扑	application.yml
mybatis-plus.global-config.dbConfig.idType	ASSIGN_ID	雪花主键	application.yml
mybatis-plus.global-config.dbConfig.logicDeleteValue	2 (未删 0)	逻辑删除值	application.yml
mybatis-plus.mapperPackage/mapperLocations	com.cpems.**.mapper,... / classpath*:mapper/**/*Mapper*.xml	扫描规则	application.yml
xss.enabled / urlPatterns	true / /system*/,/monitor*/,/tool/*	XSS 过滤	application.yml
lock4j.acquire-timeout/expire	3000/30000	获取锁超时/锁过期 (ms)	application.yml
management.endpoints.web.exposure.include	*	actuator 暴露范围	application.yml
management.endpoint.logfile.external-file	./logs/sys-console.log	日志文件端点	application.yml
captcha.type/category	MATH/CIRCLE	验证码类型	application.yml

配置键	默认/示例值	说明	位置
前端 VUE_APP_BASE_API	http://localhost:8088/autoee- iot-ems	前端后端地址	zhurong-admin- ui/.env.developme nt
前端 dev 端口	9029	webpack devServer	vue.config.js

12. 本地排障与日志定位指南

12.1 启动阶段

现象	定位手段	结论/处理
启动即退出, Communications link failure	检查 MySQL 容器与 3306	起容器、核对 root/123456、库 autoee_ems
Unable to connect to RabbitMQ / ACCESS_REFUSED	控制台 http://localhost:15672 看 用户与 vhost	创建 rabbitmq 用户、 vhost admin_vhost 并授 权
Redis NOAUTH Authentication required	用 redis-cli 带密码连	local 与 redisson 两处补 密码 difyai123456
【失败】-初始化连接 MQTT 重试 10 次	EMQX Dashboard 是否可 登录	启动 emqx; 核对 mqtt.host/admin/public ; clientId 冲突则改名
全站接口 500, InaccessibleObjectExce ption	看堆栈是否 CGLIB/反射	JDK 8 启动, 或加齐 10 项 --add-opens
xxl-job 注册失败刷日志	看 xxl.job.enabled	local 保持 false; 联调再 开
YAML Scanner @...@ 占 位符报错	additional-location 误 指向目录	只能指向具体 application-local.yml 文件

12.2 MQTT/MQ 链路

按日志关键字串联全链路（同一设备数据顺序出现）：

1. 接收消息主题：electric/voltage, 接收消息内容：...
(MyMQTTCallback)

2. 已转发数据到 `electric/voltage: {...}`
3. [开始]-接收到 RabbitMQ 的消息-进行业务处理 (MyAckReceiver)
4. 业务表/TDengine 写入、报警日志;
5. 无后续日志时去 RabbitMQ 管理台看队列是否有 `unacked/ready` 堆积:
 - `ready` 堆积: 消费者未启动或抛异常未 `ack`;
 - `unacked` 卡住: 消费线程阻塞 (并发为 1), 查最后一条业务日志;
6. 医院链路额外看: `hospital_callback_log` 表状态
(`SUCCESS/AUTH_FAIL/FAIL + error_msg + cost_time`)、
`hospital.device.data.retry.queue` 是否有消息 (重试中)、[IOT 回调]
数据重试 3 次仍失败 日志 (需人工介入)。

EMQX 侧: Dashboard → Clients (`zhurong-ems-local-client` 是否在线)、
Topics/Subscriptions (订阅是否齐全)、Websocket 工具可手工 `publish` 复现。

12.3 数据与权限

- 接口 401: `request` 未带 `token` 或 `token` 过期 → 重新登录; 看 `Authorization` 头是否 `Bearer xxx`;
- 接口返回无权限: 核对 `sys_menu.perms` 与 `@SaCheckPermission` 字符串、角色是否勾选菜单;
- 菜单不显示: 查 `/getRouters` 返回; `sys_menu` 中菜单状态、组件路径; 前端 `component` 必须能在 `src/views/` 找到, 否则路由跳转空白;
- 列表缺数据: 先确认是否数据权限过滤 (`resolveAccessibleAreas`), 再查 SQL (`p6spy/StdOutImpl` 本地已开 SQL 日志);
- 报「表不存在»: 从 `.dev/dumps/ems_full.sql` 或 数据库脚本/ 补 DDL。

12.4 日志文件与端点

- 日志框架 Logback, 配置 `classpath:logback.xml`; 控制台同时落盘 `./logs/sys-console.log` (工作目录为仓库根或 IDEA 模块运行目录);
- XXL-Job 执行日志: `./logs/xxl-job` (local 关闭时无);
- Actuator: `http://localhost:8088/autoee-iot-ems/actuator/health` (db/redis/rabbit 分项状态, `show-details=ALWAYS`)、
`/actuator/logfile` 直接取日志;
- 前端: 浏览器 Network 看 `VUE_APP_BASE_API` 拼接是否正确; 跨域问题本地不存在 (直连 8088, 生产由 Nginx 同源代理 `/prod-api/`、`/autoee-iot-ems/`)。

12.5 阅读源码建议顺序

1. `cpEmsApplication` → `application.yml` / `application-local.yml`, 建立运行时视图;
2. `MqttConfiguration` → `MyMQTTClient` → `MyMQTTCallback` → `RabbitExchangeConfig` → `MessageListenerConfig` → `MyAckReceiver`, 掌握老数据链路;
3. `HospitalCallbackController` → `HospitalDataIngestServiceImpl` → `(Auth/Parser)` → `HospitalRabbitConfig` → `HospitalDeviceDataConsumer/RetryConsumer` → `HospitalAlarmEvalServiceImpl`, 掌握新链路工程范式;
4. `SysLoginController` → `SysLoginService` → `ISysMenuService.buildMenus` 对照前端 `permission.js` / `store/modules/permission.js`, 打通权限闭环;
5. 最后按业务域包名浏览 `com.cpems.web.controller.*`, 即可快速定位任何功能。

配套文档: 03-01 开发环境搭建手册.md | 03-02 编码规范与开发约定.md